UNIVERSITY OF RWANDA

RMA- GAKO

COLLEGE OF SCIENCE AND TECHNOLOGY

SCHOOL OF ICT

COMPUTER

ENGINEERING LEVEL THREE

NAME: EDUARD GASASIRA

223026683

Name: Eduard GASASIRA 223026683
RMA - Gako
Year 3 Computer Engineering
Semester II
Assignment

State Management in flutter

① . * ~~Provider~~

Definition

State management: It is the management of the state of one or more user interface controls such as text fields, Ok buttons, radio buttons, etc.

* Provider: Is a simple and widely used state management solution in flutter.
It uses the InheritedWidget concept to share data between widgets efficiently.
Provider helps manage the application state and updates the UI whenever the state changes.

Key ideas:

- State is stored in a ChangeNotifier class
- Widgets listen to the state.
- When the state changes, UI rebuilds Automatically.

Advantages

- Easy to learn
- Recommended by flutter team
- Good for small and medium apps.

* Riverpod

Is an improved version of provider created by the same developer. It removes some limitations of provider and makes state management safer and easier to test.

Key idea

- State is stored in provider
- No need for BuildContext
- Works well with large and scalable applications.

Advantages

- Compile - time safety
- Easier testing
- Better architecture for large apps

* Bloc

Bloc stands for Business Logic Component. It separates the business logic from the UI using streams and events.

Key idea

- UI sends events
- Bloc processes events
- Bloc emits new states
- UI rebuilds when state changes.

Advantages

- Clean architecture
- Good for large and complex apps
- Suitable for team development
- More complex than provider

* Get X

Is a powerful and lightweight flutter framework for state management, dependency injection, and routing.

Key idea

- Uses reactive variables
- UI updates automatically when values changes.

Advantages

- very fast development
- less boilerplate code
- Easy to learn

Disadvantages

- Architecture is less strict compared to Bloc.

② . Table : When Each State Management is Applicable.

Situation	Recommended choice	Reason
Small applications	Provider or Getx	Simple to set up with very little "boilerplate" code to write.
Medium applications	Riverpod	offers a great balance of flexibility and scalability as the app grows.
Large / Enterprise App's	Bloc	Provides extreme predictability and a "single source of truth" for complex data.
Team projects	Bloc	Its strict rules ensure everyone on the team writes code the same way.
Fast Development	Getx	Allows for rapid prototyping because it combines states routing, and DI in one tool.
Strict Architecture	Bloc	Enforces a clear separation between UI and logic by design.

3. The **Provider** package is a wrapper around `InheritedWidget` that simplifies state management by allowing you to easily share and update data across your Flutter app.

1. **Adding Dependency**
   First, add the [Provider package]() to your project. Run this command in your terminal:

```
flutter pub add provider
```

```
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.1.2  # Use the latest version
```

2. Creating a State Class

Create a class that extends `ChangeNotifier`. This class holds your application's data and logic. Use the `notifyListeners()` method to signal when a change occurs.

```dart
import 'package:flutter/material.dart';

class CounterProvider extends ChangeNotifier {
  int _count = 0;
  int get count => _count;

  void increment() {
    _count++;
    notifyListeners(); // This triggers the UI rebuild
  }
}
```

3. Providing the State

To make the state accessible, wrap your app (or a specific part of the widget tree) with a `ChangeNotifierProvider`. This is typically done in the `main()` function.

```
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => CounterProvider(),
      child: const MyApp(),
    ),
  );
}
```

4. Accessing the State

You can access the data in your widgets using several methods

- **`context.watch<T>()`**: Listens for changes and rebuilds the widget when the state updates.

- **`Consumer<T>`**: A widget that rebuilds only its own builder function when the state changes, which is more efficient for large widget trees.
- **`context.read<T>()`**: Accesses the state without listening for changes (ideal for calling functions in button presses). [3, 6, 8, 9, 10]

```
// Example using Consumer
Consumer<CounterProvider>(
  builder: (context, counter, child) {
    return Text('Count: ${counter.count}');
  },
)
```

5. Updating the State

To modify the state, call the methods defined in your state class. Use `context.read<T>()` to trigger these actions because you generally don't need to "listen" to the provider just to call a function.

```
ElevatedButton(
  onPressed: () {
    context.read<CounterProvider>().increment();
  },
  child: const Text('Increment'),
)
```

6. How UI Rebuild Happens

The UI rebuild occurs through the **notifyListeners()** mechanism.

Flow of UI Update

1. User presses a button
2. Provider method updates the state
3. `notifyListeners()` is called
4. Widgets listening to the provider rebuild automatically

Example:

```dart
class HomeScreen extends StatelessWidget {

  @override
  Widget build(BuildContext context) {

    int count = context.watch<CounterProvider>().count;

    return Scaffold(
      appBar: AppBar(title: Text("Provider Example")),

      body: Center(
        child: Text(
          "$count",
          style: TextStyle(fontSize: 40),
        ),
      ),

      floatingActionButton: FloatingActionButton(
        onPressed: () {
          context.read<CounterProvider>().increment();
        },
        child: Icon(Icons.add),
      ),
    );
  }
}
```

What happens here

- `context.watch()` listens for changes
- When `increment()` is called
- `notifyListeners()` triggers rebuild
- UI automatically updates with the new counter value